MODUL PRAKTIKUM

PRAKTIKUM DASAR PEMROGRAMAN

Modul 3

Operator, Ekspresi, Konversi Tipe Data, dan Tracing Program

Disusun oleh:

Dr. Desta Sandya Prasvita, S.Komp., M.Kom.

PROGRAM STUDI S1 INFORMATIKA

UPN VETERAN JAKARTA

2026

Modul ini melanjutkan pemahaman mahasiswa tentang data dan variabel pada Modul 2. Fokus praktikum adalah menggunakan operator untuk membentuk ekspresi, memahami urutan evaluasi, menangani pembagian bilangan bulat, melakukan konversi tipe data, serta menelusuri perubahan nilai variabel secara sistematis.

I. Pendahuluan Modul

1. Identitas Modul

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Semester	1
Modul	3
Judul	Operator, Ekspresi, Konversi Tipe Data, dan Tracing Program
Bahasa Pemrograman	C
Alokasi Waktu	1 × 170 menit
Metode	Demonstrasi, latihan terbimbing, eksperimen, debugging, tracing, dan latihan mandiri
Prasyarat	Modul 1 dan Modul 2

2. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

- Mahasiswa mampu menggunakan operator aritmatika, relasional, dan logika dalam ekspresi program C.
- Mahasiswa mampu menentukan urutan evaluasi ekspresi berdasarkan prioritas dan asosiativitas operator.
- Mahasiswa mampu menerapkan konversi tipe data secara tepat dalam perhitungan.

II. Persiapan Praktikum

1. Persiapan Folder Praktikum

Buat struktur folder berikut:

```
Praktikum_Dasar_Pemrograman/
└── Modul_03/
    ├── latihan_01.c
    ├── latihan_02.c
    ├── latihan_03.c
    ├── tracing.c
    ├── eksperimen.c
    ├── debugging.c
    └── tugas_modul_03.c
```

Gunakan nama berkas yang:

- menggunakan huruf kecil;
- tidak mengandung spasi;
- menggunakan garis bawah _;
- memiliki ekstensi .c.

III. Konsep Dasar

1. Operator, Operand, dan Ekspresi

Operator

Operator adalah simbol yang digunakan untuk melakukan operasi terhadap suatu nilai.

Contoh operator:

```
+
-
*
/
%
```

Operand

Operand adalah nilai atau variabel yang dikenai operasi.

Contoh:

```
hasil = angka_1 + angka_2;
```

Pada pernyataan tersebut:

- + merupakan operator;
- angka_1 dan angka_2 merupakan operand;
- angka_1 + angka_2 merupakan ekspresi;
- = merupakan operator penugasan.

Ekspresi

Ekspresi adalah kombinasi nilai, variabel, operator, dan pemanggilan fungsi yang menghasilkan suatu nilai.

Contoh:

```
panjang * lebar
(nilai_uts + nilai_uas) / 2.0
harga * jumlah
(total_detik % 3600) / 60
```

Ekspresi dapat sederhana atau kompleks. Semakin kompleks ekspresi, semakin penting penggunaan tanda kurung dan nama variabel perantara.

2. Operator Aritmatika

Operator	Operasi	Contoh
+	Penjumlahan	a + b
-	Pengurangan	a - b
*	Perkalian	a * b
/	Pembagian	a / b
%	Sisa pembagian	a % b
+	Unary positif	+a
-	Unary negatif	-a

Contoh program:

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    int b = 3;

    printf("Penjumlahan : %d\n", a + b);
    printf("Pengurangan : %d\n", a - b);
    printf("Perkalian   : %d\n", a * b);
    printf("Pembagian   : %d\n", a / b);
    printf("Sisa bagi   : %d\n", a % b);

    return 0;
}
```

Keluaran:

```
Penjumlahan : 13
Pengurangan : 7
Perkalian   : 30
Pembagian   : 3
Sisa bagi   : 1
```

3. Pembagian Bilangan Bulat

Pembagian dalam bahasa C bergantung pada tipe operand.

Pembagian dua bilangan bulat

```
int hasil = 5 / 2;
```

Nilai hasil adalah:

```
2
```

Bagian pecahan tidak dibulatkan, tetapi dibuang.

Contoh lain:

```
printf("%d\n", 9 / 5);
```

Keluaran:

```
1
```

Pembagian bilangan pecahan

```
double hasil = 5.0 / 2.0;
```

Nilainya adalah:

```
2.5
```

Salah satu operand saja sudah cukup dibuat bertipe pecahan:

```
double hasil_1 = 5.0 / 2;
double hasil_2 = 5 / 2.0;
```

Keduanya menghasilkan:

```
2.5
```

Kesalahan yang sering terjadi

```
int jumlah = 5;
int banyak_data = 2;
double rata_rata;

rata_rata = jumlah / banyak_data;
```

Walaupun rata_rata bertipe double, operasi berikut dilakukan terlebih dahulu:

```
jumlah / banyak_data
```

Karena kedua operand bertipe int, hasil pembagiannya adalah 2. Setelah itu nilai 2 dikonversi menjadi 2.0.

Perbaikan:

```
rata_rata = (double) jumlah / banyak_data;
```

atau:

```
rata_rata = jumlah / (double) banyak_data;
```

4. Operator Modulo

Operator % menghasilkan sisa pembagian dua bilangan bulat.

Contoh:

```
10 % 3
```

menghasilkan:

```
1
```

karena:

```
10 = 3 × 3 + 1
```

Contoh lain:

Ekspresi	Hasil
8 % 2	0
9 % 2	1
17 % 5	2
25 % 10	5
60 % 60	0

Operator modulo berguna untuk:

- menentukan sisa pembagian;
- memisahkan jam, menit, dan detik;
- menentukan digit terakhir;
- memeriksa pola kelipatan;
- menentukan bilangan genap atau ganjil setelah materi seleksi dipelajari.

Contoh mengambil digit terakhir:

```
int bilangan = 578;
int digit_terakhir = bilangan % 10;
```

Nilai digit_terakhir adalah:

```
8
```

Operator % digunakan untuk operand bilangan bulat. Jangan menggunakannya secara langsung pada float atau double.

5. Operator Penugasan

Operator = digunakan untuk memberikan nilai kepada variabel.

```
jumlah = 10;
```

Pernyataan tersebut berarti nilai 10 disimpan ke dalam variabel jumlah.

Operator penugasan bukan operator perbandingan.

```
jumlah = 10;
```

berarti memberikan nilai. Sementara itu:

```
jumlah == 10
```

berarti membandingkan apakah nilai jumlah sama dengan 10.

Operator == akan digunakan lebih banyak pada modul kondisional.

6. Operator Penugasan Majemuk

Operator penugasan majemuk mempersingkat penulisan ekspresi.

Bentuk panjang	Bentuk ringkas
a = a + b;	a += b;
a = a - b;	a -= b;
a = a * b;	a *= b;
a = a / b;	a /= b;
a = a % b;	a %= b;

Contoh:

```
int saldo = 100000;

saldo += 50000;
saldo -= 25000;
```

Perubahan nilai:

Pernyataan	Nilai saldo
int saldo = 100000;	100000
saldo += 50000;	150000
saldo -= 25000;	125000

Contoh program:

```
#include <stdio.h>

int main(void)
{
    int nilai = 10;

    printf("Nilai awal : %d\n", nilai);

    nilai += 5;
    printf("Setelah += : %d\n", nilai);

    nilai *= 2;
    printf("Setelah *= : %d\n", nilai);

    nilai -= 4;
    printf("Setelah -= : %d\n", nilai);

    return 0;
}
```

7. Operator Increment dan Decrement

Increment

Operator increment menambah nilai variabel sebesar satu.

```
nilai++;
```

atau:

```
++nilai;
```

Decrement

Operator decrement mengurangi nilai variabel sebesar satu.

```
nilai--;
```

atau:

```
--nilai;
```

Digunakan sebagai pernyataan terpisah

Apabila digunakan sebagai pernyataan terpisah, hasil akhirnya sama.

```
nilai++;
```

dan:

```
++nilai;
```

keduanya menambah nilai sebesar satu.

Digunakan dalam ekspresi

Perbedaannya terlihat ketika operator digunakan di dalam ekspresi.

Post-increment:

```
int a = 5;
int b = a++;
```

Hasilnya:

```
b = 5
a = 6
```

Nilai lama a digunakan terlebih dahulu, kemudian a ditambah satu.

Pre-increment:

```
int a = 5;
int b = ++a;
```

Hasilnya:

```
a = 6
b = 6
```

Nilai a ditambah terlebih dahulu, kemudian digunakan.

Untuk mahasiswa pemula, sebaiknya increment dan decrement digunakan sebagai pernyataan terpisah:

```
jumlah++;
```

Hindari ekspresi yang sulit dibaca:

```
hasil = nilai++ + ++nilai;
```

Ekspresi seperti itu juga dapat menghasilkan perilaku yang tidak terdefinisi karena satu variabel diubah beberapa kali dalam satu ekspresi tanpa urutan yang aman.

8. Operator Relasional

Operator relasional digunakan untuk membandingkan dua nilai.

Operator	Arti
==	Sama dengan
!=	Tidak sama dengan
>	Lebih besar
<	Lebih kecil
>=	Lebih besar atau sama dengan
<=	Lebih kecil atau sama dengan

Dalam bahasa C, hasil perbandingan direpresentasikan sebagai:

- 1 untuk benar;
- 0 untuk salah.

Contoh:

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    int b = 5;

    printf("a > b  : %d\n", a > b);
    printf("a < b  : %d\n", a < b);
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);

    return 0;
}
```

Keluaran:

```
a > b  : 1
a < b  : 0
a == b : 0
a != b : 1
```

Pada modul ini, operator relasional baru digunakan untuk mengamati hasil ekspresi. Penerapannya dalam keputusan program dibahas pada Modul 4.

9. Operator Logika

Operator	Arti
&&	AND
||	OR
!	NOT

Operator AND

Ekspresi bernilai benar apabila kedua kondisi benar.

```
(a > 0) && (b > 0)
```

Operator OR

Ekspresi bernilai benar apabila minimal satu kondisi benar.

```
(a > 0) || (b > 0)
```

Operator NOT

Membalik nilai logika.

```
!(a > 0)
```

Contoh:

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    int b = -2;

    printf("%d\n", (a > 0) && (b > 0));
    printf("%d\n", (a > 0) || (b > 0));
    printf("%d\n", !(a > 0));

    return 0;
}
```

Keluaran:

```
0
1
0
```

Gunakan tanda kurung pada setiap kondisi agar ekspresi mudah dibaca.

10. Prioritas Operator

Bahasa C memiliki aturan urutan evaluasi operator.

Versi sederhana yang digunakan pada modul ini adalah:

Prioritas	Operator
1	()
2	Unary +, unary -, !, ++, --, casting
3	*, /, %
4	+, -
5	<, <=, >, >=
6	==, !=
7	&&
8	||
9	=, +=, -=, *=, /=, %=

Contoh:

```
int hasil = 2 + 3 * 4;
```

Perkalian dikerjakan terlebih dahulu:

```
2 + 12 = 14
```

Dengan tanda kurung:

```
int hasil = (2 + 3) * 4;
```

Hasilnya:

```
5 × 4 = 20
```

Gunakan tanda kurung untuk memperjelas maksud

Meskipun compiler mengetahui prioritas operator, pembaca program belum tentu langsung memahaminya.

Kurang jelas:

```
total = harga + harga * pajak / 100;
```

Lebih jelas:

```
total = harga + ((harga * pajak) / 100);
```

Atau menggunakan variabel perantara:

```
nilai_pajak = harga * pajak / 100.0;
total = harga + nilai_pajak;
```

Variabel perantara sering membuat program lebih mudah ditelusuri dan diperbaiki.

11. Konversi Tipe Data

Konversi tipe data adalah perubahan suatu nilai dari satu tipe menjadi tipe lain.

Konversi dapat terjadi:

- secara implisit;
- secara eksplisit.

a. Konversi Implisit

Konversi implisit dilakukan secara otomatis oleh compiler.

Contoh:

```
int jumlah = 10;
double hasil = jumlah;
```

Nilai 10 yang bertipe int dikonversi menjadi:

```
10.0
```

Contoh lain:

```
int a = 5;
double b = 2.0;
double hasil = a + b;
```

Nilai a dikonversi menjadi double, sehingga hasilnya:

```
7.0
```

b. Konversi Eksplisit atau Type Casting

Konversi eksplisit ditentukan oleh programmer.

Bentuk umum:

```
(tipe_data) nilai
```

Contoh:

```
int jumlah = 5;
int banyak_data = 2;

double rata_rata = (double) jumlah / banyak_data;
```

Hasilnya:

```
2.5
```

Tanpa casting:

```
double rata_rata = jumlah / banyak_data;
```

Hasilnya:

```
2.0
```

c. Konversi dari Pecahan ke Bilangan Bulat

```
double nilai = 8.75;
int hasil = (int) nilai;
```

Nilai hasil menjadi:

```
8
```

Bagian pecahan dibuang, bukan dibulatkan.

Contoh lain:

```
double nilai = -8.75;
int hasil = (int) nilai;
```

Nilainya menjadi:

```
-8
```

Konversi menuju bilangan bulat bergerak menuju nol.

d. Kehilangan Informasi

Konversi dari tipe yang lebih luas atau lebih presisi ke tipe yang lebih sempit dapat menghilangkan informasi.

```
double nilai = 92.875;
int nilai_bulat = nilai;
```

Nilai nilai_bulat menjadi:

```
92
```

Compiler dapat memberikan warning untuk konversi tertentu. Oleh karena itu, konversi harus dilakukan dengan sengaja dan dipahami dampaknya.

12. Batas dan Risiko Operasi

a. Pembagian dengan nol

Ekspresi berikut tidak boleh dilakukan:

```
hasil = angka / 0;
```

Pembagian bilangan bulat dengan nol dapat menyebabkan program berhenti atau menghasilkan perilaku yang tidak terdefinisi.

Karena seleksi belum dibahas secara penuh, pastikan data uji pada modul ini tidak menggunakan pembagi nol.

b. Overflow

Setiap tipe data memiliki batas nilai.

Apabila hasil operasi melebihi batas tipe data, dapat terjadi overflow.

Contoh yang berisiko:

```
int angka_besar = 2000000000;
int hasil = angka_besar * 2;
```

Gunakan nilai yang berada dalam jangkauan tipe data dan pilih tipe yang sesuai.

c. Operator ^ bukan perpangkatan

Dalam bahasa C:

```
2 ^ 3
```

bukan berarti dua pangkat tiga.

Operator ^ merupakan operator bitwise XOR. Perpangkatan dibahas menggunakan fungsi dari pustaka matematika pada materi yang sesuai.

13. Tracing Program

Tracing adalah proses menelusuri eksekusi program langkah demi langkah.

Tujuan tracing:

- memahami alur program;
- mengetahui perubahan nilai variabel;
- memprediksi output;
- menemukan kesalahan logika;
- memeriksa kesesuaian algoritma dengan source code.

Perhatikan program berikut:

```
#include <stdio.h>

int main(void)
{
    int a = 7;
    int b = 3;
    int sisa;
    double hasil;

    a += 2;
    b++;
    sisa = a % b;
    hasil = (double) a / b;

    printf("a     = %d\n", a);
    printf("b     = %d\n", b);
    printf("sisa  = %d\n", sisa);
    printf("hasil = %.2f\n", hasil);

    return 0;
}
```

Trace table

Langkah	Pernyataan	a	b	sisa	hasil
1	int a = 7;	7	–	–	–
2	int b = 3;	7	3	–	–
3	a += 2;	9	3	–	–
4	b++;	9	4	–	–
5	sisa = a % b;	9	4	1	–
6	hasil = (double) a / b;	9	4	1	2.25

Keluaran:

```
a     = 9
b     = 4
sisa  = 1
hasil = 2.25
```

Aturan mengisi trace table

- baca program dari atas ke bawah;
- catat nilai awal setiap variabel;
- kerjakan satu pernyataan pada satu waktu;
- hanya ubah variabel yang terpengaruh;
- evaluasi bagian dalam tanda kurung terlebih dahulu;
- perhatikan tipe setiap operand;
- jangan menebak hasil akhir tanpa menuliskan proses.

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Kalkulator Aritmatika Dasar

Buat berkas: latihan_01.c. Tuliskan program berikut:

```
#include <stdio.h>

int main(void)
{
    int angka_1;
    int angka_2;
    int penjumlahan;
    int pengurangan;
    int perkalian;
    int pembagian_bulat;
    int sisa_bagi;
    double pembagian_real;

    printf("Masukkan angka pertama: ");
    scanf("%d", &angka_1);

    printf("Masukkan angka kedua selain nol: ");
    scanf("%d", &angka_2);
```

```
    penjumlahan = angka_1 + angka_2;
    pengurangan = angka_1 - angka_2;
    perkalian = angka_1 * angka_2;
    pembagian_bulat = angka_1 / angka_2;
    sisa_bagi = angka_1 % angka_2;
    pembagian_real = (double) angka_1 / angka_2;

    printf("\nHASIL OPERASI\n");
    printf("Penjumlahan       : %d\n", penjumlahan);
    printf("Pengurangan       : %d\n", pengurangan);
    printf("Perkalian         : %d\n", perkalian);
    printf("Pembagian bulat   : %d\n", pembagian_bulat);
    printf("Sisa pembagian    : %d\n", sisa_bagi);
    printf("Pembagian riil    : %.2f\n", pembagian_real);

    return 0;
}
```

Contoh input

```
Angka pertama: 17
Angka kedua: 5
```

Keluaran yang diharapkan

```
Penjumlahan       : 22
Pengurangan       : 12
Perkalian         : 85
Pembagian bulat   : 3
Sisa pembagian    : 2
Pembagian riil    : 3.40
```

Pertanyaan analisis

1. Mengapa nilai angka_2 tidak boleh nol?
2. Apa perbedaan pembagian_bulat dan pembagian_real?
3. Mengapa casting ditempatkan sebelum angka_1?
4. Apakah casting pada angka_2 akan menghasilkan nilai yang sama?
5. Berapa hasil apabila inputnya 5 dan 2?
6. Berapa hasil apabila inputnya 2 dan 5?
7. Apa hubungan pembagian bulat dengan sisa pembagian?

2. Praktikum Terbimbing 2 — Konversi Total Detik

Buat berkas: latihan_02.c. Program mengubah total detik menjadi jam, menit, dan detik.

```
#include <stdio.h>

int main(void)
{
    int total_detik;
    int jam;
    int menit;
    int detik;
    int sisa_detik;

    printf("Masukkan total detik: ");
    scanf("%d", &total_detik);

    jam = total_detik / 3600;
    sisa_detik = total_detik % 3600;
    menit = sisa_detik / 60;
    detik = sisa_detik % 60;

    printf("\nHASIL KONVERSI\n");
    printf("%d detik setara dengan:\n", total_detik);
    printf("%d jam\n", jam);
    printf("%d menit\n", menit);
    printf("%d detik\n", detik);

    return 0;
}
```

Contoh

Input:

```
3725
```

Proses:

```
jam         = 3725 / 3600 = 1
sisa_detik  = 3725 % 3600 = 125
menit       = 125 / 60 = 2
detik       = 125 % 60 = 5
```

Keluaran:

```
1 jam
2 menit
5 detik
```

Trace table

Langkah	total_detik	jam	sisa_detik	menit	detik
Input	3725	–	–	–	–
Hitung jam	3725	1	–	–	–
Hitung sisa	3725	1	125	–	–
Hitung menit	3725	1	125	2	–
Hitung detik	3725	1	125	2	5

Pertanyaan analisis

1. Mengapa jam menggunakan pembagian?
2. Mengapa sisa_detik menggunakan modulo?
3. Mengapa detik dihitung dari sisa_detik, bukan langsung melalui pembagian?
4. Apa hasil untuk input 60?
5. Apa hasil untuk input 3600?
6. Apa hasil untuk input 3661?
7. Bagaimana program menangani input negatif? Apakah hasil tersebut bermakna?

3. Praktikum Terbimbing 3 — Menghitung Nilai Akhir

Buat berkas: latihan_03.c. Program menghitung nilai akhir berdasarkan komponen:

- tugas: 20%;
- kuis: 10%;
- UTS: 30%;
- UAS: 40%.

```
#include <stdio.h>

int main(void)
{
    const double BOBOT_TUGAS = 0.20;
    const double BOBOT_KUIS = 0.10;
    const double BOBOT_UTS = 0.30;
    const double BOBOT_UAS = 0.40;

    double nilai_tugas;
    double nilai_kuis;
    double nilai_uts;
    double nilai_uas;
    double nilai_akhir;

    printf("Masukkan nilai tugas: ");
    scanf("%lf", &nilai_tugas);

    printf("Masukkan nilai kuis: ");
    scanf("%lf", &nilai_kuis);

    printf("Masukkan nilai UTS: ");
    scanf("%lf", &nilai_uts);
```

```
    printf("Masukkan nilai UAS: ");
    scanf("%lf", &nilai_uas);

    nilai_akhir =
        (nilai_tugas * BOBOT_TUGAS) +
        (nilai_kuis * BOBOT_KUIS) +
        (nilai_uts * BOBOT_UTS) +
        (nilai_uas * BOBOT_UAS);

    printf("\nNilai akhir: %.2f\n", nilai_akhir);

    return 0;
}
```

Contoh input

Komponen	Nilai
Tugas	80
Kuis	90
UTS	75
UAS	85

Perhitungan manual

```
Tugas = 80 × 0.20 = 16.00
Kuis  = 90 × 0.10 =  9.00
UTS   = 75 × 0.30 = 22.50
UAS   = 85 × 0.40 = 34.00
                          -----
Nilai akhir             = 81.50
```

Pertanyaan analisis

1. Mengapa bobot menggunakan tipe double?
2. Mengapa bobot dideklarasikan sebagai konstanta?
3. Berapa jumlah seluruh bobot?
4. Apa akibatnya apabila jumlah bobot tidak sama dengan 1.0?
5. Mengapa ekspresi nilai akhir ditulis dalam beberapa baris?
6. Apakah tanda kurung pada setiap komponen wajib?
7. Bagaimana cara menguji kebenaran nilai akhir?

V. Eksperimen dan Tracing

1. Latihan Tracing

Buat berkas: tracing.c. Jangan langsung menjalankan program berikut. Prediksi hasilnya terlebih dahulu.

```
#include <stdio.h>

int main(void)
{
    int x = 8;
    int y = 3;
    int z;
    double hasil;

    x += 4;
    y *= 2;
    z = x % y;
    hasil = (double) x / y;
    x--;
    y++;

    printf("x     = %d\n", x);
    printf("y     = %d\n", y);
    printf("z     = %d\n", z);
    printf("hasil = %.2f\n", hasil);

    return 0;
}
```

Lengkapi trace table:

Langkah	Pernyataan	x	y	z	hasil
1	int x = 8;	8	–	–	–
2	int y = 3;				
3	x += 4;				
4	y *= 2;				
5	z = x % y;				
6	hasil = (double) x / y;				
7	x--;				
8	y++;				

Setelah tabel selesai:

- tuliskan prediksi output;
- kompilasi program;
- jalankan program;
- bandingkan hasil aktual dengan hasil tracing;
- perbaiki tabel apabila terdapat kesalahan.

2. Eksperimen

Buat berkas: eksperimen.c. Gunakan program dasar berikut:

```
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 2;
    double hasil;

    hasil = a / b;

    printf("Hasil: %.2f\n", hasil);

    return 0;
}
```

Lakukan eksperimen satu per satu.

No.	Perubahan	Prediksi	Hasil Aktual
1	Gunakan hasil = a / b;		
2	Gunakan hasil = (double) a / b;		
3	Gunakan hasil = a / (double) b;		
4	Ubah a menjadi 5.0 bertipe double		
5	Simpan 8.75 ke variabel int		
6	Bandingkan 2 + 3 * 4 dan (2 + 3) * 4		
7	Bandingkan a++ dan ++a dalam pernyataan terpisah		
8	Gunakan a += 3 sebagai pengganti a = a + 3		
9	Tampilkan hasil a > b, a == b, dan a != b		
10	Ubah salah satu pembagi menjadi nol—analisis saja, jangan dijalankan		

Untuk setiap eksperimen:

- tuliskan prediksi;
- ubah satu bagian program;
- kompilasi dengan warning aktif;
- catat warning atau error;
- jalankan apabila aman;
- jelaskan penyebab hasil;
- kembalikan program ke kondisi awal.

VI. Debugging Challenge

1. Debugging Challenge

Program berikut dimaksudkan untuk menghitung subtotal, nilai diskon, dan total pembayaran. Program memiliki kesalahan sintaks, tipe data, input, format output, dan logika.

```
#include <stdio.h>

int main(void)
{
    int harga_satuan;
    int jumlah_barang
    int persen_diskon = 10;
    int subtotal;
    double nilai_diskon;
    double total_bayar;

    printf("Masukkan harga satuan: ");
    scanf("%d", harga_satuan);

    printf("Masukkan jumlah barang: ");
    scanf("%f", &jumlah_barang);

    subtotal = harga_satuan + jumlah_barang;
    nilai_diskon = persen_diskon / 100 * subtotal;
    total_bayar = subtotal - nilai_diskon;

    printf("\nSubtotal      : %.2f\n", subtotal);
    printf("Nilai diskon : %d\n", nilai_diskon);
    printf("Total bayar  : %.2f\n", total_bayar);

    return 0;
}
```

Tugas

1. Salin program ke debugging.c.
2. Kompilasi sebelum melakukan perbaikan.
3. Catat setiap error dan warning.
4. Identifikasi kesalahan satu per satu.
5. Perbaiki source code.
6. Uji menggunakan: harga satuan 20000, jumlah barang 3, diskon 10%.
7. Bandingkan hasil program dengan perhitungan manual.
8. Jelaskan mengapa ekspresi diskon pada program awal menghasilkan nilai yang salah.

Hasil yang diharapkan

```
Subtotal      : Rp60000
Nilai diskon  : Rp6000.00
Total bayar   : Rp54000.00
```

Tabel debugging

No.	Bagian yang Salah	Jenis Kesalahan	Penyebab	Perbaikan
1				
2				
3				
4				
5				
6				
7				

VII. Latihan Mandiri

1. Melengkapi Ekspresi

Lengkapi program berikut:

```
#include <stdio.h>

int main(void)
{
    int total_detik = 7325;
    int jam;
    int menit;
    int detik;
    int sisa;

    jam = total_detik ____ 3600;
    sisa = total_detik ____ 3600;
    menit = sisa ____ 60;
    detik = sisa ____ 60;

    printf("%d jam, %d menit, %d detik\n",
           jam,
           menit,
           detik);

    return 0;
}
```

2. Konversi Hari

Buat program yang menerima total hari, kemudian mengubahnya menjadi:

- minggu;
- sisa hari.

Contoh:

```
Masukkan total hari: 25

25 hari setara dengan:
3 minggu
4 hari
```

Gunakan pembagian bilangan bulat dan modulo.

3. Rata-Rata Nilai

Buat program yang menerima lima nilai bilangan bulat dan menghitung rata-ratanya dalam bentuk pecahan.

Contoh:

```
Nilai 1: 80
Nilai 2: 75
Nilai 3: 90
Nilai 4: 85
Nilai 5: 88

Jumlah nilai : 418
Rata-rata    : 83.60
```

Program wajib menggunakan casting agar hasil rata-rata tidak kehilangan bagian pecahan.

4. Membalik Bilangan Tiga Digit

Buat program yang menerima bilangan bulat tiga digit dan menampilkan setiap digit serta bentuk terbaliknya.

Contoh:

```
Masukkan bilangan tiga digit: 572

Digit ratusan : 5
Digit puluhan : 7
Digit satuan  : 2
Bilangan balik: 275
```

Petunjuk:

```
ratusan = bilangan / 100
puluhan = (bilangan / 10) % 10
satuan  = bilangan % 10
```

Bilangan balik dapat dihitung menggunakan:

```
(satuan × 100) + (puluhan × 10) + ratusan
```

Pada modul ini, diasumsikan pengguna memasukkan bilangan tiga digit yang valid.

5. Distribusi Kue Ulang Tahun

Keluaran program pada soal ini akan diperiksa secara ketat: harus sama persis karakter demi karakter dengan Contoh Output.

Sebuah acara ulang tahun ingin membagikan kue kepada anak-anak yang hadir secara merata. Buat program C yang menerima total kue dan jumlah anak, lalu menghitung pembagian kue secara utuh, sisa kue, dan rata-rata kue per anak dalam bentuk desimal.

Format Input

Dua baris bilangan bulat: total kue dan jumlah anak.

Batasan

- $0 \leq$ total kue ≤ 10000;
- $1 \leq$ jumlah anak ≤ 100.

Contoh 1

Input:

```
50
8
```

Output:

```
========================================
       DISTRIBUSI KUE ULANG TAHUN
========================================
Total kue           : 50
Jumlah anak         : 8
----------------------------------------
Kue per anak (utuh) : 6
Sisa kue            : 2
Rata-rata per anak  : 6.25
========================================
```

Contoh 2

Input:

```
100
4
```

Output:

```
========================================
       DISTRIBUSI KUE ULANG TAHUN
========================================
Total kue           : 100
Jumlah anak         : 4
----------------------------------------
Kue per anak (utuh) : 25
Sisa kue            : 0
Rata-rata per anak  : 25.00
========================================
```

Contoh 3

Input:

```
17
5
```

Output:

```
========================================
       DISTRIBUSI KUE ULANG TAHUN
========================================
Total kue           : 17
Jumlah anak         : 5
----------------------------------------
Kue per anak (utuh) : 3
Sisa kue            : 2
Rata-rata per anak  : 3.40
========================================
```

Petunjuk Pendekatan

- gunakan variabel bertipe int untuk total_kue dan jumlah_anak;
- kue per anak (utuh) dihitung dengan operator / antara dua int;
- sisa kue dihitung dengan operator % antara dua int;
- rata-rata per anak harus berbentuk desimal, gunakan casting (double) pada salah satu operand sebelum pembagian;
- gunakan %.2f untuk mencetak rata-rata agar selalu tampil dua angka di belakang koma;
- ketiga baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

VIII. Pengujian Program

1. Pengujian Program

Setiap program harus diuji menggunakan beberapa data.

Contoh pengujian program konversi waktu

No.	Total Detik	Expected Output	Actual Output	Status
1	0	0 jam, 0 menit, 0 detik		
2	59	0 jam, 0 menit, 59 detik		
3	60	0 jam, 1 menit, 0 detik		
4	3600	1 jam, 0 menit, 0 detik		
5	3661	1 jam, 1 menit, 1 detik		
6	7325	2 jam, 2 menit, 5 detik		

Aspek yang diperiksa

- program dapat dikompilasi;
- tidak ada warning penting;
- operator yang digunakan sesuai;
- hasil pembagian bilangan bulat benar;
- hasil modulo benar;
- casting digunakan pada lokasi yang tepat;
- format output sesuai;
- hasil sama dengan perhitungan manual;
- perubahan nilai variabel sesuai trace table.

IX. Tugas Praktikum

1. Tugas Praktikum Modul 3 — Program Ringkasan Aktivitas Belajar

Buat program C untuk menghitung dan menampilkan ringkasan aktivitas belajar seorang mahasiswa dalam satu minggu.

Input

Program menerima:

- jumlah sesi belajar;
- total durasi belajar dalam menit;
- jumlah halaman yang dibaca;
- jumlah latihan yang diselesaikan;
- target latihan mingguan.

Proses

Program menghitung:

```
jam belajar          = total menit ÷ 60
sisa menit           = total menit % 60
rata-rata per sesi   = total menit ÷ jumlah sesi
persentase latihan   = jumlah latihan ÷ target latihan × 100
```

Perhitungan rata-rata dan persentase harus menghasilkan nilai pecahan.

Contoh input

```
Jumlah sesi belajar        : 5
Total durasi belajar       : 425 menit
Jumlah halaman dibaca      : 120
Latihan diselesaikan       : 18
Target latihan mingguan    : 20
```

Keluaran yang diharapkan

```
================================================
          RINGKASAN AKTIVITAS BELAJAR
================================================
Jumlah sesi              : 5 sesi
Total waktu belajar      : 7 jam 5 menit
Halaman dibaca           : 120 halaman
Rata-rata per sesi       : 85.00 menit
Latihan diselesaikan     : 18 dari 20
Persentase penyelesaian  : 90.00%
================================================
```

Analisis IPO

Mahasiswa wajib melengkapi tabel berikut sebelum menulis source code.

Komponen	Uraian
Input	
Process	
Output	

Ketentuan source code

Program wajib:

- menggunakan #include <stdio.h>;
- menggunakan int main(void);
- menggunakan nama variabel deskriptif;
- menggunakan tipe data sesuai kebutuhan;
- menggunakan operator / dan %;
- menggunakan type casting pada perhitungan yang membutuhkan pecahan;
- menggunakan minimal satu operator penugasan majemuk atau increment secara wajar;
- menampilkan hasil pecahan dengan dua angka di belakang koma;
- menggunakan konstanta apabila terdapat nilai tetap;
- menyertakan komentar identitas dan deskripsi;
- dikompilasi dengan warning aktif;
- tidak menghasilkan error;
- diuji menggunakan minimal lima test case;
- disertai trace table untuk satu data uji;
- dapat dijelaskan oleh mahasiswa.

Batasan Modul 3

Karena materi seleksi belum dibahas secara penuh, program mengasumsikan:

- jumlah sesi lebih besar dari nol;
- target latihan lebih besar dari nol;
- seluruh input berupa bilangan tidak negatif;
- pengguna memasukkan tipe data yang benar.

Kondisi tersebut harus dituliskan sebagai keterbatasan program. Validasi akan ditambahkan pada Modul 4.

Header komentar

```
/*
Nama        :
NIM         :
Kelas       :
Modul       : 3
Deskripsi   : Program menghitung ringkasan aktivitas belajar.
*/
```

Berkas yang dikumpulkan

- source code: tugas_modul_03_nim.c
- analisis input–process–output;
- daftar variabel dan tipe datanya;
- penjelasan ekspresi yang digunakan;
- trace table untuk satu data uji;
- tabel pengujian minimal lima kasus;
- tangkapan layar kompilasi;
- tangkapan layar hasil eksekusi;
- pernyataan penggunaan AI.

2. Kecepatan Rata-rata Perjalanan

Buat program C yang menerima jarak tempuh dan waktu tempuh, lalu menghitung kecepatan rata-rata dalam km/jam. Keluaran program dinilai secara ketat: harus sama persis karakter demi karakter, termasuk spasi.

Format Input

Dua baris bilangan bulat: jarak tempuh dalam kilometer, dan waktu tempuh dalam menit.

Batasan

- $1 \leq \text{jarak} \leq 1000$ (km);
- $1 \leq \text{waktu} \leq 600$ (menit).

Contoh 1

Input:

```
120
90
```

Output:

```
========================================
          KECEPATAN RATA-RATA
========================================
Jarak tempuh   : 120 km
Waktu tempuh   : 90 menit
----------------------------------------
Kecepatan      : 80.00 km/jam
========================================
```

Contoh 2

Input:

```
45
30
```

Output:

```
========================================
          KECEPATAN RATA-RATA
========================================
Jarak tempuh   : 45 km
Waktu tempuh   : 30 menit
----------------------------------------
Kecepatan      : 90.00 km/jam
========================================
```

Contoh 3

Input:

```
100
150
```

Output:

```
========================================
          KECEPATAN RATA-RATA
========================================
Jarak tempuh   : 100 km
Waktu tempuh   : 150 menit
----------------------------------------
Kecepatan      : 40.00 km/jam
========================================
```

Ketentuan

- gunakan variabel bertipe int untuk jarak dan waktu;
- rumus kecepatan: (jarak / (double) waktu) * 60.0, gunakan casting eksplisit agar hasil pembagian tidak terpotong;

- gunakan %.2f untuk mencetak kecepatan agar selalu tampil dua angka di belakang koma;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output;
- keluaran diperiksa apa adanya (karakter demi karakter), sehingga kesalahan satu spasi tetap dianggap tidak sesuai.

Simpan sebagai tugas_modul_03_soal2_nim.c dan sertakan sebagai lampiran tambahan bersama tugas utama.

X. Kesalahan Umum dan Standar Penulisan

1. Kesalahan yang Sering Terjadi

a. Mengharapkan hasil pecahan dari pembagian int

Salah:

```
int jumlah = 5;
int banyak = 2;
double rata_rata = jumlah / banyak;
```

Hasil:

```
2.00
```

Benar:

```
double rata_rata = (double) jumlah / banyak;
```

Hasil:

```
2.50
```

b. Melakukan casting setelah pembagian

Tidak efektif:

```
double hasil = (double) (jumlah / banyak);
```

Pembagian bilangan bulat sudah terjadi sebelum casting.

Benar:

```
double hasil = (double) jumlah / banyak;
```

c. Menggunakan % pada bilangan pecahan

Salah:

```
double a = 10.5;
double b = 3.0;

double hasil = a % b;
```

Operator % digunakan untuk tipe bilangan bulat.

d. Salah menggunakan prioritas operator

Berpotensi salah:

```
rata_rata = nilai_1 + nilai_2 + nilai_3 / 3.0;
```

Hanya nilai_3 yang dibagi tiga.

Benar:

```
rata_rata = (nilai_1 + nilai_2 + nilai_3) / 3.0;
```

e. Menggunakan = untuk perbandingan

```
hasil = a = b;
```

Pernyataan tersebut memberikan nilai b kepada a, kemudian nilainya disimpan ke hasil.

Perbandingan sama dengan menggunakan:

```
a == b
```

Penerapannya dibahas lebih lanjut pada Modul 4.

f. Menggunakan ^ untuk perpangkatan

Salah:

```
luas = sisi ^ 2;
```

Operator ^ bukan operator pangkat. Untuk luas persegi:

```
luas = sisi * sisi;
```

g. Membuat ekspresi terlalu kompleks

Kurang mudah dibaca:

```
hasil = a + b * c / d % e - f;
```

Lebih baik menggunakan tanda kurung atau variabel perantara:

```
hasil_perkalian = b * c;
hasil_pembagian = hasil_perkalian / d;
hasil = a + (hasil_pembagian % e) - f;
```

h. Mengubah variabel beberapa kali dalam satu ekspresi

Hindari:

```
hasil = nilai++ + ++nilai;
```

Gunakan pernyataan terpisah:

```
nilai++;
hasil = nilai;
nilai++;
```

2. Standar Penulisan Ekspresi

Gunakan spasi

Disarankan:

```
total_biaya = harga_satuan * jumlah_barang;
```

Kurang rapi:

```
total_biaya=harga_satuan*jumlah_barang;
```

Gunakan tanda kurung

Disarankan:

```
rata_rata = (nilai_1 + nilai_2 + nilai_3) / 3.0;
```

Gunakan variabel perantara

Disarankan:

```
subtotal = harga_satuan * jumlah_barang;
nilai_pajak = subtotal * PERSENTASE_PAJAK;
total_bayar = subtotal + nilai_pajak;
```

Hindari angka tanpa penjelasan

Kurang baik:

```
hasil = total * 0.11;
```

Lebih baik:

```
const double TARIF_PAJAK = 0.11;

hasil = total * TARIF_PAJAK;
```

XI. Pernyataan penggunaan AI

Tidak menggunakan AI:

```
Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan
AI generatif.
```

Menggunakan AI:

```
Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan
[operator, ekspresi, error, atau bagian lain]. Hasil tersebut saya
periksa melalui tracing, kompilasi, pengujian, dan perhitungan manual.
```

Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari https://gcc.gnu.org/onlinedocs/

International Organization for Standardization. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C. ISO.

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/languages/cpp

Savitch, W. (2015). Problem solving with C++ (9th ed.). Pearson Education.